Projeto e Análise de Algoritmos

Programação Dinâmica

Universidade Federal do Amazonas
Instituto de Computação

Roteiro

- Programação Dinâmica
  - Problemas de Otimização
  - Multiplicação de Matrizes
  - Princípios de Programação Dinâmica
  - Maior Subseqüência Comum
  - Distância de Edição

Problemas de Otimização

- Problemas que apresentam várias soluções, cada uma com um valor (custo) associado.
- Procura-se a solução com valor ótimo (mínimo ou máximo)
- Uma solução geralmente apresenta uma estrutura: ela é composta de uma sequência de escolhas. Tais escolhas devem ser feitas para se chegar à solução ótima.

Programação Dinâmica

- Caracterizar a estrutura de uma solução ótima
- Definir recursivamente o valor de uma solução ótima
- Calcular o valor da solução ótima em processo bottom-up (de problemas menores para maiores)
- Construir uma solução ótima a partir de informações calculadas



Exemplo: Multiplicação de cadeias de matrizes

Multiplicação de Matrizes

- Duas matrizes *A:n×m e B:m×k* podem ser multiplicadas usando *nmk* multiplicações escalares.

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{pmatrix} = \begin{pmatrix} \cdots & \cdots & \cdots \\ \cdots & c_{22} & \cdots \\ \cdots & \cdots & \cdots \end{pmatrix} \qquad c_{i,j} = \sum_{l=1}^{m} a_{i,l} \cdot b_{l,j}$$

- Problema: Computar o produto de muitas matrizes de forma eficiente.
- A multiplicação de matrizes é associativa
  - *(AB)C = A(BC)*

Multiplicação de Matrizes (2)

- A ordem ditada pelos parênteses afeta o custo da multiplicação
- Considere $A \times B \times C \times D$, *onde*
  - A:30×1, B :1×40, C :40×10, D :10×25
- Custos:
  - $(AB)C)D = 1200 + 12000 + 7500 = 20700$
  - $(AB)(CD) = 1200 + 10000 + 30000 = 41200$
  - $A((BC)D) = 400 + 250 + 750 = 1400$
- Procuramos uma sequência ótima para multiplicar
  - A_1 X A_2X … X A_n onde A_i é uma matriz d_{i-1} X d_i

Multiplicação de Matrizes (3)

- Seja *M(i,j)* o número mínimo de multiplicações necessárias $\prod_{k=i}^{j} A_k$
- Observações importantes:
  - Os parênteses mais externos da cadeia de matrizes, dividem a sequência em algum *k*, ($i \leq k < j$): $(A_i \ldots A_k)(A_{k+1} \ldots A_j)$
  - A solução ótima da sequência *(i,j)* é formada por soluções ótimas das duas subsequências *(i,k)* e *(k+1,j)*

Multiplicação de Matrizes (4)

- Se todos os *k* são verificados. Temos a recorrência:

$$M(i,i) = 0$$

$$M(i,j) = \min_{i \le k < j} \left\{ M(i,k) + M(k+1,j) + d_{i-1} d_k d_j \right\}$$

- M(1,n)= **$\Omega(2^n)$**
- A implementação recursiva direta é exponencial. Muita computação redundante ocorre.
- No entanto, o número de subproblemas distintos é consideravelmente menor

Quantos subproblemas eu tenho ?

Considere i=1 e j=n

$$M(i,i) = 0$$

$$M(i,j) = \min_{i \le k < j} \left\{ M(i,k) + M(k+1,j) + d_{i-1} d_k d_j \right\}$$

	1	2	3	4	5
1	0				
2		0			
3			0		
4				0	
5					0

	1	2	3	4	5
1	0		2		
2		0			
3			0		
4				0	
5					0

Como funciona a programação dinâmica

Problemas da forma:

$M(1,1)=M(2,2)=...=M(n,n)=0$

São computados quase de graça!

Subproblemas na forma M(i,i+1)

- Podem ser computados em função dos problemas na forma M(i,i)

$$M(i,i) = 0$$

$$M(i,j) = \min_{i \le k < j} \left\{ M(i,k) + M(k+1,j) + d_{i-1} d_k d_j \right\}$$

- M(1,2)=M(1,1)+M(2,2)+d...
- Único valor possível para k é 1

Agora posso computar M(i,i+2)!

$$M(1,3) = \min_{1 \le k < 3} \{ M(1,k) + M(k+1,3) + d_0 . d_k . d_3 \}$$

$$M(1,3) = \min \begin{cases} M(1,1) + M(2,3) + d_0 . d_1 . d_3 = 1 : (A_1).(A_2 . A_3) \\ M(1,2) + M(3,3) + d_0 . d_2 . d_3 = 2 : (A_1 . A_2).(A_3) \end{cases}$$

- Agora temos 2 valores possíveis para k (todos em função de valores já computados

Multiplicação de Matrizes (5)

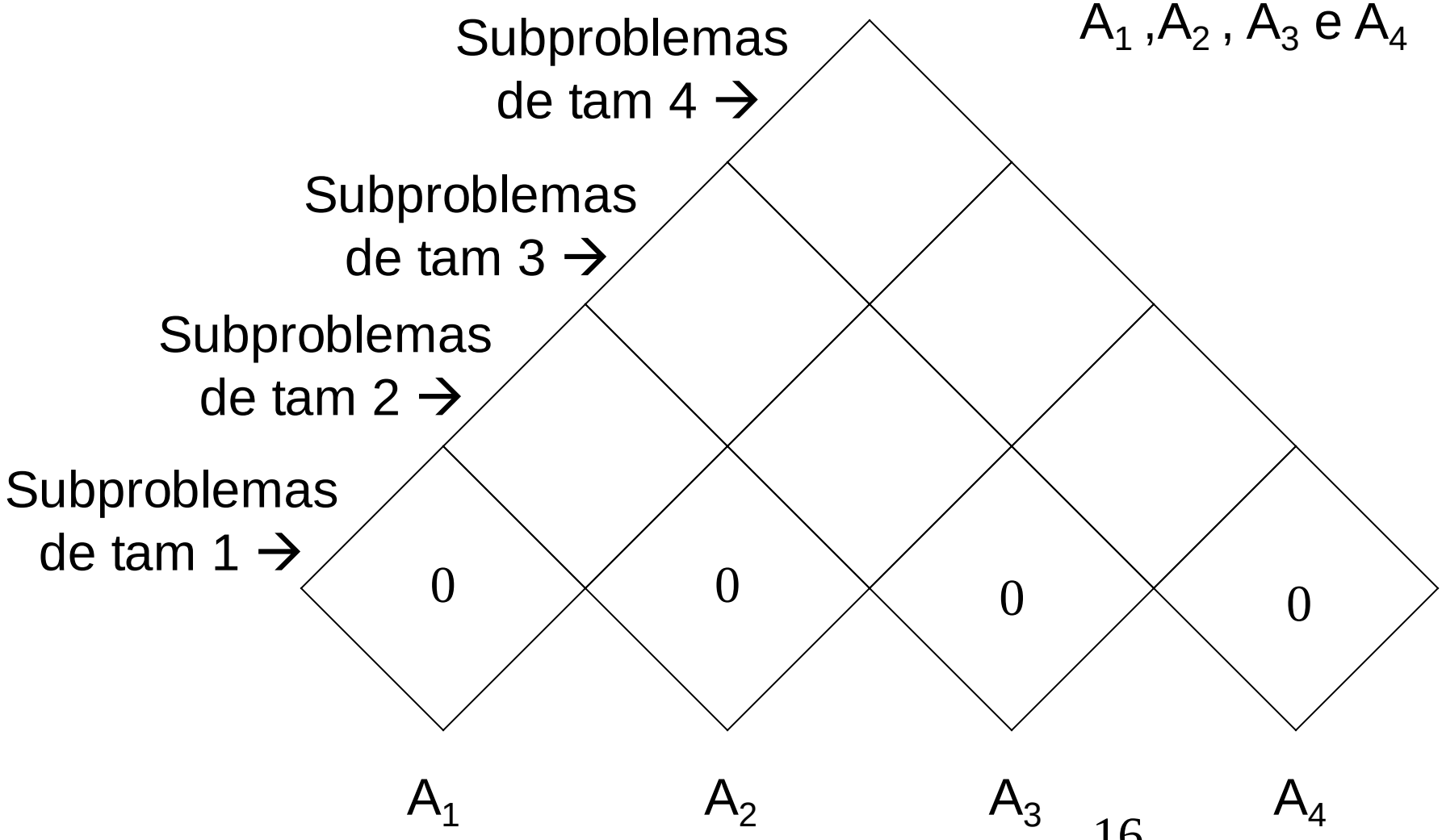
$$M(1,4) = \min_{1 \le k < 4} \{ M(1,k) + M(k+1,4) + d_0 . d_k . d_4 \}$$

$$M(1,4) = \min \begin{cases} M(1,1) + M(2,4) + d_0 . d_1 . d_4 = 1 : (A_1).(A_2 . A_3 . A_4) \\ M(1,2) + M(3,4) + d_0 . d_2 . d_4 = 2 : (A_1 . A_2).(A_3 . A_4) \\ M(1,3) + M(4,4) + d_0 . d_3 . d_4 = 3 : (A_1 . A_2 . A_3).(A_4) \end{cases}$$

$$M(1,3) = \min_{1 \le k < 3} \{ M(1,k) + M(k+1,3) + d_1 . d_k . d_3 \}$$

$$M(1,3) = \min \begin{cases} M(1,1) + M(2,3) + d_1 . d_3 . d_4 = 1 : (A_1).(A_2 . A_3) \\ M(1,2) + M(3,3) + d_0 . d_2 . d_4 = 2 : (A_1 . A_2).(A_3) \end{cases}$$

Multiplicação de Matrizes (6)



Multiplicação de Matrizes (7)

- Assim, podemos armazenar as soluções ótimas de cada $\Theta(n^2)$ subproblema e reutiliza-las na solução dos problemas maiores
- Estas soluções podem ser armazenadas em um arranjo *M[1..n,1..n]*
- O custo computar de cada M[i,j] é O(n)

Multiplicação de Matrizes (8)

```
Ordem(d0…dn)
1   para i←1 até n faça
2       M[i,i] ← 0
3   para l←2 até n faça
4       para i←1 até n-l+1 faça
5           j ← i+l-1
•           M[i,j] ← ∞
∀           para k←i até j-l faça
•               q ← M[i,k]+M[k+1,j]+d_{i-1}d_k d_j
9               se q < M[i,j] então
•               M[i,j] ← q
•               c[i,j] ← k
12  retorne M, c
```

Multiplicação de Matrizes (9)

- Ao fim da execução
  - *M[1,n] contêm o valor da solução ótima*
  - *c as escolhas ótimas de k para cada subproblema*
  - *M guarda os resultados obtidos a cada solução!*
- Executar para *d = [10, 20, 3, 5, 30,4]*

$$M(i,i) = 0$$

$$M(i,j) = \min_{i \le k < j} \left\{ M(i,k) + M(k+1,j) + d_{i-1} d_k d_j \right\}$$

Executar para *d = [10, 20, 3, 5, 30,4]*

$$M(i,i) = 0$$

$$M(i,j) = \min_{i \le k < j} \left\{ M(i,k) + M(k+1,j) + d_{i-1} d_k d_j \right\}$$

	1	2	3	4	5
1	0	600	750	1950	XXXX
2		0	300	2250	
3			0	450	660
4				0	600
5					0

	1	2	3	4	5
1	1	2	1	2	
2		2	3	3	
3			3	3	3
4	K=1			4	5
5	400				5

Multiplicação de Matrizes (10)

- Tempo de Execução : $O(n^3)$
- Melhoramos de expoencial para polinomial !!!!

Memoizaçao

- Computar solução usando solução recursiva inicial, mas guardando os valores da matriz já computados para evitar processamento desnecessário.
- Adaptar a solução recursiva para tabular as soluções intermediárias
- As chamadas recursivas continuam, mas não precisam necessariamente executar as operações custosas
- No exemplo das matrizes
  - Iniciar os elementos de *M* com ∞ e executar **Busca-Sequência**(d, i, j)

Memoizaçao (2)

```
Busca-Sequência(d,i,j)
1   se M[i,j] < ∞ então
2      retorna m[i,j]
3   se i=j então
•      m[i,j] ←0
5   senão para k ← i até j-1 faça
•           q ← Busca-Sequência(d,i,k)+
       Busca-Sequência(d,k+1,j)+d_{i-1}d_k d_j
7           se q < M[i,j] então
8              M[i,j] ← q
9   retorne M[i,j]
```

Programação Dinâmica

- Em geral, para aplicar PD, alguns aspectos devem ser considerados:
  - 1. Sub-estrutura ótima: Uma solução ótima para o problema deve ser composta de soluções ótimas para os seus subproblemas

Programação Dinâmica (2)

- 2. Escrever uma recorrência para determinar o valor da solução ótima

$M_{\text{ótima}} = \min_{\text{sobre todas as escolhas de } k}$ {(Soma de $M_{\text{ótima}}$ de todos os subproblemas que resultam da escolha de *k*)+ (custo associado com a escolha de *k*)}

- Mostrar que o número de instâncias distintas dos subproblemas é limitado por um polinômio.

Programação Dinâmica (3)

- 3. Computar o valor da solução ótima de maneira *bottom-up*, de forma que todos os sub-resultados já estejam pré-computados.
- Verificar se é possível reduzir espaço eliminando sub-resultados que não são mais necessários.
- Construir a solução ótima a partir da informação pré-computada

Maior Subseqüência Comum

- São dadas duas cadeias de caracteres
- Deseja-se estabelecer o quão semelhante elas são
  - Comparação de seqüências de DNA/Proteínas
  - Correção ortográfica
- Uma medida de semelhança é o comprimento da Maior Subseqüência Comum (MSC) entre os dois

MSC: Definição

- *Z* é uma subseqüência de X, se é possível gerar Z removendo alguns caracteres de X.
  - *X* =“ACGGTTA”
  - *Y*=“CGTAT”
  - MSC(*X*,*Y*) = “CGTA” ou “CGTT”

MSC: Subestrutura Ótima

- Seja X_m="$x_1 x_2 \ldots x_m$" e Y_n="$y_1 y_2 \ldots y_n$"
  - Se $x_m = y_n$, inclua este caractere no início de Z e encontre MSC (X_{m-1}, Y_{n-1})
  - Se $x_m \neq y_n$,
    - Pular um caractere de X ou de Y
    - Decidir o que fazer comparando MSC(X_m, Y_{n-1}) e MSC(X_{m-1}, Y_n)

MSC: Recorrência

- Seja c[i,j] = MSC(X_i, Y_j)

$$c[i,j] = \begin{cases} 0 & se\ i = 0\ ou\ j = 0 \\ c[i-1, j-1] + 1 & se\ i, j > 0\ e\ x_i = y_j \\ \max\{c[i, j-1], c[i-1, j]\} & se\ i, j > 0\ e\ x_i \neq y_j \end{cases}$$

MSC: Algoritmo

```
MCS(X, Y, m, n)
1  para i←1 até m faça
2     c[i,0] ← 0
3  para j←0 até n faça
4     c[0,j] ← 0
5  para i←1 até m faça
6     para j←1 até n faça
7        se x_i = y_j então
•           c[i,j] ← c[i-1,j-1]+1
9           b[i,j] ← "copia"
10       senão se c[i-1,j] ≥ c[i,j-1] então
•                c[i,j] ← c[i-1,j]
•                b[i,j] ← "avança x"
13             senão
•                c[i,j] ← c[i,j-1]
•                b[i,j] ← "avança y"
16 retorne c, b
```

Exercício

- Mostre como a tabela seria construída para as seqüências ACGTACGGGTA e GTCGATTTACG
- Qual o tamanho da LCS encontrada ?
- Qual seria a LCS ?

Distância de edição

- Permitir inserção, remoção ou substituição de caracteres no texto
- Número de erros é conhecido como ***distância de edição*** ou ***distância de Levenshtein***
- Objetivo pode ser capturar erros de escrita, ocr ou simplesmente ajudar um usuário que não sabe como escrever uma dada palavra

Especificação recursiva

$$d(x_m, y_n) \begin{cases} d(x_{m-1}, y_{n-1}) & x[m] = y[n], m > 0, n > 0 \\ 1 + \min(d(x_{m-1}, y_n); d(x_m, y_{n-1}); d(x_{m-1}, y_{n-1}) & x[m] \neq y[n], m > 0, n > 0 \\ m & n = 0 \\ n & m = 0 \end{cases}$$



Calcule a distancia de edicao para:

micail gorbachov

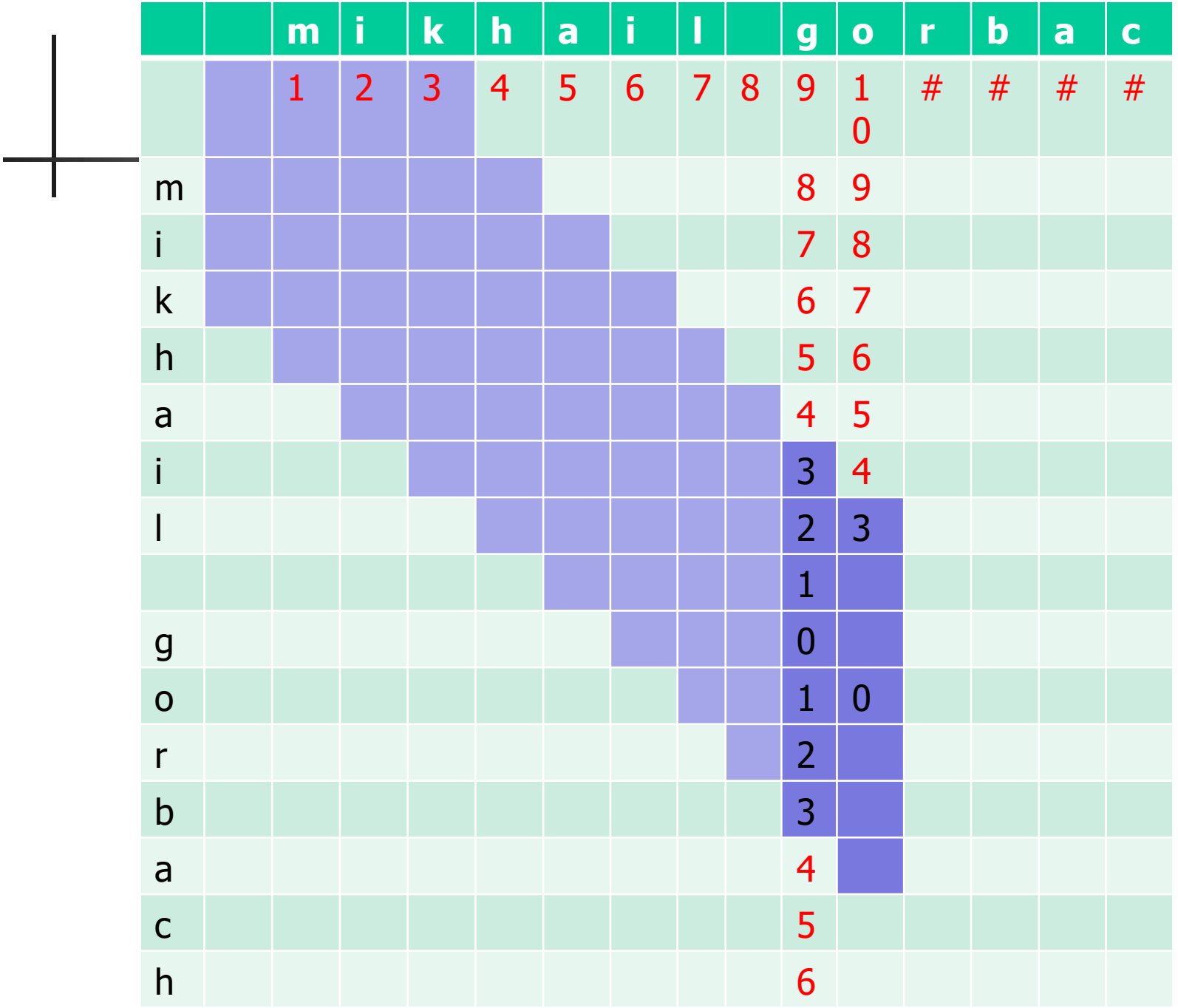
mikhail gorbachev

		m	i	c	a	i	l		g	o	r	b	a	c	h
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
m	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13
i	2	1	0	1	2	3	4	5	6						
k	3	2	1	1	2	3	4	5	6						
h	4	3	2	2	2	3	4	5	6						
a	5	4	3	3	2	3	4	5	6						
i	6	5	4	4	3	2	3	4	5						
l	7	6	5	5	4	3	2	3	4						
	8	7	6	6	5	4	3	2	3						
g	9	8	7	7	6	5	4	3	2						
o	10	9	8	8	7	6	5	4	3						
r	11	10	9	9	8	7	6	5	4						
b	12	11	10	10	9	8	7	6	5						
a	13	12	11	11	10	9	8	7	6						
c	14	13	12	12	11	10	9	8	7						
h	15	14	13	13	12	11	10	9	8						

		m	i	c	a	i	l		g	o	r	b	a	c	h
										9	10	11	12	13	14
m										8					
i										7					
k										7					
h										7					
a										7					
i										6					
l										5					
										4					
g										3					
o										2					
r										3					
b										4					
a										5					
c										6					
h										7					

E se eu limitar a distância de edição

Limitando a distância, podemos economizar memória e processamento.


m i k h a i l g o r b a c
1 2 3 4 5 6 7 8 9 10 # # # #
m 8 9
i 7 8
k 6 7
h 5 6
a 4 5
i 3 4
l 2 3
1
g 0
o 1 0
r 2
b 3
a 4
c 5
h 6

Letras iguais

3	
2	
1	
0	
1	
2	
3	